

Лабораторна робота №4

з теми «Написання інтеграційних тестів»

Виконала:

ст. гр. ІП-22-1

Чемна (Микитин) О. І.

Перевірив:

Храбатин Р. І.

Мета: Навчитися створювати інтеграційні тести, розібратися в існуючих інструментах для створення інтеграційних тестів та навчитися їх обирати.

Короткі теоретичні відомості:

Інтеграційне тестування використовується для перевірки взаємодії між різними частинами програмної системи.

Одним із найпоширеніших прикладів такого тестування є перевірка API, оскільки саме API забезпечує обмін даними між клієнтом, сервером та іншими сервісами. Якщо API працює некоректно, це може вплинути на весь застосунок і погіршити користувацький досвід.

API-тестування дозволяє перевіряти правильність відповідей сервера, статус-коди, структуру даних і поведінку різних HTTP-методів. Найчастіше для таких перевірок використовуються методи GET, POST, PUT і DELETE. Метод GET застосовується для отримання даних, POST — для створення нового запису, PUT — для оновлення існуючого запису, а DELETE — для видалення даних.

Для виконання API-запитів у JavaScript можна використовувати різні клієнти, наприклад Axios, Got, Node-fetch, Ку або Superagent.

У цій роботі використовується Axios, оскільки він має простий синтаксис, зручно працює з промісами та добре підходить для навчальних інтеграційних тестів. Для запуску тестів використовується Jest, який дозволяє організовувати перевірки у тестові набори за допомогою describe, test та expect.

Завдання:

Завдання перше:

Переглянути наявні клієнти для роботи з API запитом. Обрати один з них. Провести необхідні маніпуляції для їх встановлення. Написати простий запит GET для перевірки роботи інструменту.

Завдання друге:

Обрати існуючий безкоштовний API сервіс або використати свій, якщо такий наявний, можна також його створити. Перед питаннями для самоконтролю буде надано ряд посилань. Написати по 5 тестових наборів для

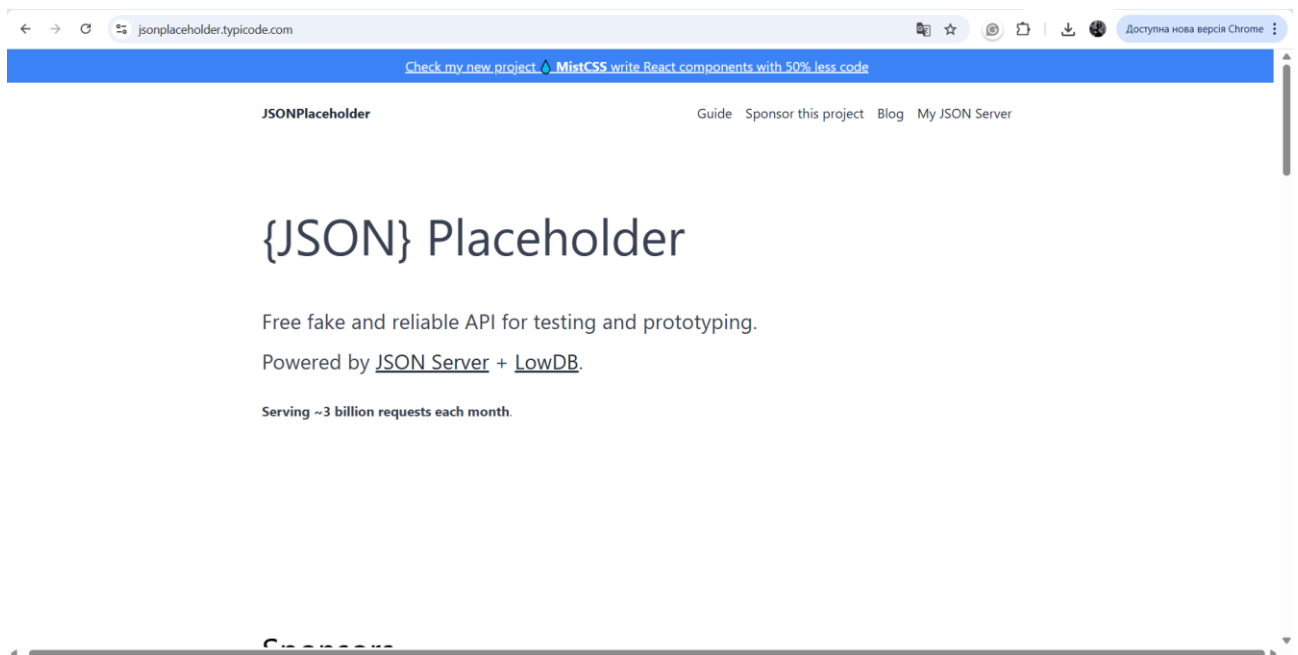
перевірки методів GET(двох варіацій: всі записи та конкретний), POST, PUT, DELETE.

Завдання третє:

Завантажити код на власний/навчальний репозиторій GitHub, або інший схожий ресурс та поділитися посиланням.

Виконання:

Для виконання роботи я обрала API-сервіс JSONPlaceholder:



Я обрала цей сервіс, оскільки він є безкоштовним, не потребує реєстрації, не вимагає API-ключа та дозволяє перевіряти основні HTTP-методи: GET, POST, PUT і DELETE.

Спочатку я створила окремий каталог для виконання лабораторної роботи:

```
oksanka@debian:~/Documents/automated_testing$ mkdir integration-testing  
oksanka@debian:~/Documents/automated_testing$ cd integration-testing
```

Після цього я ініціалізувала Node.js-проект:

```

• oksanka@debian:~/Documents/automated_testing/integration-testing$ npm init -y
Wrote to /home/oksanka/Documents/automated_testing/integration-testing/package.json:

{
  "name": "integration-testing",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC"
}

• oksanka@debian:~/Documents/automated_testing/integration-testing$

```

Далі я встановила Jest для запуску тестів:

```

• oksanka@debian:~/Documents/automated_testing/integration-testing$ npm install --save-dev jest
npm warn deprecated inflight@1.0.6: This module is not supported, and leaks memory. Do not use it. Check out lru-cache if you want a good and tested way to coalesce async requests by a key value, which is much more comprehensive and powerful.
npm warn deprecated glob@7.2.3: Old versions of glob are not supported, and contain widely publicized security vulnerabilities, which have been fixed in the current version. Please update. Support for old versions may be purchased (at exorbitant rates) by contacting i@izs.me
npm warn deprecated glob@10.5.0: Old versions of glob are not supported, and contain widely publicized security vulnerabilities, which have been fixed in the current version. Please update. Support for old versions may be purchased (at exorbitant rates) by contacting i@izs.me

added 293 packages, and audited 294 packages in 19s

44 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
• oksanka@debian:~/Documents/automated_testing/integration-testing$

```

Після цього я встановила Axios для виконання API-запитів:

```

• oksanka@debian:~/Documents/automated_testing/integration-testing$ npm install axios

added 24 packages, and audited 318 packages in 4s

50 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
• oksanka@debian:~/Documents/automated_testing/integration-testing$

```

Оскільки я використовувала синтаксис `import`, я також встановила Babel-залежності:

```

• oksanka@debian:~/Documents/automated_testing/integration-testing$ npm install --save-dev @babel/core @babel/preset-env babel-jest

added 91 packages, and audited 409 packages in 5s

54 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
• oksanka@debian:~/Documents/automated_testing/integration-testing$

```

Далі я створила файл `.babelrc`. У файл `.babelrc` я додала такі налаштування:

```

B .babelrc U X
B .babelrc > ...
1 {
2   "presets": ["@babel/preset-env"]
3 }
4

```

Після цього я відкрила файл `package.json` і змінила секцію `scripts`, щоб тести можна було запускати командою `npm test`:

```

6   "scripts": {
7     "test": "jest"
8   },

```

Далі я створила файл для інтеграційних тестів. Спочатку я написала простий GET-запит для перевірки роботи Axios та доступності API:

```

B .babelrc U package.json U api.test.js U X
api.test.js > ...
1   import axios from "axios"; 71.8k (gzipped: 26.5k)
2
3   axios.get("https://jsonplaceholder.typicode.com/posts").then((response) => {
4     console.log(response.data);
5   });

```

```

oksanka@debian:~/Documents/automated_testing/integration-testing$ node api.test.js
[
  {
    userId: 1,
    id: 1,
    title: 'sunt aut facere repellat provident occaecati excepturi optio reprehenderit',
    body: 'quia et suscipit\n' +
      'suscipit recusandae consequuntur expedita et cum\n' +
      'reprehenderit molestiae ut ut quas totam\n' +
      'nostrum rerum est autem sunt rem eveniet architecto'
  },
  {
    userId: 1,
    id: 2,
    title: 'qui est esse',
    body: 'est rerum tempore vitae\n' +

```

Після перевірки простого запиту я перейшла до написання повноцінних API-тестів за допомогою Jest. У роботі я створила п'ять тестових наборів: для отримання всіх записів, отримання одного запису, створення запису, оновлення запису та видалення запису.

Повний код файлу `api.test.js` мав такий вигляд:

```

B .babelrc U package.json U api.test.js U X Search integration-testing — api.test.js - integration-testing - Visual S
api.test.js > ...
1   import axios from "axios"; 71.8k (gzipped: 26.5k)
2
3   const BASE_URL = "https://jsonplaceholder.typicode.com";
4
5   describe("GET /posts", () => {
6     let response;
7
8     beforeAll(async () => {
9       response = await axios.get(`${BASE_URL}/posts`);
10    });
11
12    test("should return status code 200", () => {
13      expect(response.status).toBe(200);
14    });
15
16    test("should return an array of posts", () => {
17      expect(Array.isArray(response.data)).toBe(true);
18    });
19
20    test("should return non-empty list of posts", () => {
21      expect(response.data.length).toBeGreaterThan(0);
22    });
23  });
24

```

```

25 describe("GET /posts/1", () => {
26     let response;
27
28     beforeAll(async () => {
29         response = await axios.get(`${BASE_URL}/posts/1`);
30     });
31
32     test("should return status code 200", () => {
33         expect(response.status).toBe(200);
34     });
35
36     test("should return post with id 1", () => {
37         expect(response.data.id).toBe(1);
38     });
39
40     test("should contain required post fields", () => {
41         expect(response.data).toHaveProperty("userId");
42         expect(response.data).toHaveProperty("id");
43         expect(response.data).toHaveProperty("title");
44         expect(response.data).toHaveProperty("body");
45     });
46 });
47

```

```

48 describe("POST /posts", () => {
49     let response;
50
51     beforeAll(async () => {
52         response = await axios.post(`${BASE_URL}/posts`, {
53             title: "Test book review",
54             body: "This is a test post created during integration testing.",
55             userId: 1,
56         });
57     });
58
59     test("should return status code 201", () => {
60         expect(response.status).toBe(201);
61     });
62
63     test("should return created post title", () => {
64         expect(response.data.title).toBe("Test book review");
65     });
66
67     test("should return created post body", () => {
68         expect(response.data.body).toBe(
69             "This is a test post created during integration testing.",
70         );
71     });
72
73     test("should return created post userId", () => {
74         expect(response.data.userId).toBe(1);
75     });
76 });
77

```

```

78 describe("PUT /posts/1", () => {
79     let response;
80
81     beforeAll(async () => {
82         response = await axios.put(`${BASE_URL}/posts/1`, {
83             id: 1,
84             title: "Updated test book review",
85             body: "This post was updated during integration testing.",
86             userId: 1,
87         });
88     });
89

```

```

89
90     test("should return status code 200", () => {
91         expect(response.status).toBe(200);
92     });
93
94     test("should return updated title", () => {
95         expect(response.data.title).toBe("Updated test book review");
96     });
97
98     test("should return updated body", () => {
99         expect(response.data.body).toBe(
100             "This post was updated during integration testing.",
101         );
102     });
103
104     test("should keep post id equal to 1", () => {
105         expect(response.data.id).toBe(1);
106     });
107 });

```

```

108
109 describe("DELETE /posts/1", () => {
110     let response;
111
112     beforeAll(async () => {
113         response = await axios.delete(`${BASE_URL}/posts/1`);
114     });
115
116     test("should return status code 200", () => {
117         expect(response.status).toBe(200);
118     });
119
120     test("should return empty object after deleting post", () => {
121         expect(response.data).toEqual({});
122     });
123 });

```

Після написання тестів я запустила їх у терміналі командою:

```

• oksanka@debian:~/Documents/automated_testing/integration-testing$ npm test

> integration-testing@1.0.0 test
> jest

PASS ./api.test.js
  GET /posts
    ✓ should return status code 200 (2 ms)
    ✓ should return an array of posts (1 ms)
    ✓ should return non-empty list of posts
  GET /posts/1
    ✓ should return status code 200
    ✓ should return post with id 1
    ✓ should contain required post fields (1 ms)
  POST /posts
    ✓ should return status code 201
    ✓ should return created post title (1 ms)
    ✓ should return created post body
    ✓ should return created post userId (1 ms)
  PUT /posts/1
    ✓ should return status code 200 (1 ms)
    ✓ should return updated title (1 ms)
    ✓ should return updated body
    ✓ should keep post id equal to 1
  DELETE /posts/1
    ✓ should return status code 200 (1 ms)
    ✓ should return empty object after deleting post (1 ms)

Test Suites: 1 passed, 1 total
Tests:       16 passed, 16 total
Snapshots:   0 total
Time:        1.4 s
Ran all test suites.
• oksanka@debian:~/Documents/automated_testing/integration-testing$

```

У результаті запуску Jest виконав усі створені інтеграційні тести. Загалом я створила п'ять тестових наборів для перевірки основних API-операцій.

Після успішного запуску тестів я підготувала проєкт до завантаження на GitHub. У мене раніше уже був ініціалізований Git-репозиторій для виконання цих робіт.

Тому першим ділом я перевірила стан файлів:

```
oksanka@debian:~/Documents/automated_testing$ git status
На гілці main
Ваша гілка не відрізняється від "origin/main".

Невідстежувані файли:
  (скористайтесь "git add <файл>...", щоб додати до майбутнього коміту)
    integration-testing/

нічого не додано до коміту, але є невідстежувані файли (скористайтесь "git add" для відстежування)
oksanka@debian:~/Documents/automated_testing$
```

Далі я додала всі файли до індексації Git:

```
oksanka@debian:~/Documents/automated_testing$ git status
На гілці main
Ваша гілка не відрізняється від "origin/main".

Зміни, додані до майбутнього коміту:
  (використовуйте "git restore --staged <файл>...", щоб видалити з індексу)
    новий файл:    integration-testing/.babelrc
    новий файл:    integration-testing/.gitignore
    новий файл:    integration-testing/api.test.js
    новий файл:    integration-testing/package-lock.json
    новий файл:    integration-testing/package.json

oksanka@debian:~/Documents/automated_testing$
```

Після цього я створила коміт:

```
oksanka@debian:~/Documents/automated_testing$ git commit -m "Add integration API tests for lab 4"
[main bd8c945] Add integration API tests for lab 4
5 files changed, 6283 insertions(+)
create mode 100644 integration-testing/.babelrc
create mode 100644 integration-testing/.gitignore
create mode 100644 integration-testing/api.test.js
create mode 100644 integration-testing/package-lock.json
create mode 100644 integration-testing/package.json
oksanka@debian:~/Documents/automated_testing$
```

Потім я завантажила проєкт на GitHub:

```
oksanka@debian:~/Documents/automated_testing$ git push -u origin main
Username for 'https://github.com': OksanaMykytyn
Password for 'https://OksanaMykytyn@github.com':
Перерахування об'єктів: 9, готово.
Підрахунок об'єктів: 100% (9/9), готово.
Дельта компресія з використанням до 4 потоків
Компресія об'єктів: 100% (6/6), готово.
Запис об'єктів: 100% (8/8), 48.82 KiB | 5.42 MiB/c, готово.
Всього 8 (дельта 1), повторно використано 0 (дельта 0), повторно використано пакунків 0
(з 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/OksanaMykytyn/automated_testing.git
  9808a00..1b0be38  main -> main
гілку "main" налаштовано на відстежування "origin/main".
oksanka@debian:~/Documents/automated_testing$
```

Посилання на GitHub:

https://github.com/OksanaMykytyn/automated_testing

Висновок: під час лабораторної роботи я навчилася створювати інтеграційні API-тести за допомогою Jest та Axios. Я обрала сервіс JSONPlaceholder, оскільки він дозволяє безкоштовно перевіряти основні HTTP-методи без реєстрації та API-ключа. Я налаштувала Node.js-проект, встановила Jest, Axios і Babel. Під час виконання роботи я створила п'ять тестових наборів для перевірки методів GET, POST, PUT і DELETE. Я перевірила статус-коди відповідей, структуру отриманих даних, створення нового запису, оновлення існуючого запису та видалення запису.